Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251214**
Nama Lengkap	**Raffiel Gideon Nyolo Nyolo**
Minggu ke / Materi	**06 / Struktur Kontrol Perulangan**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

A. Definisi Perulangan

Jalannya suatu program dapat diatur secara sekuensial, percabangan, perulangan, maupun kombinasi dari ketiganya. Pengaturan tersebut biasa disebut sebagai struktur kontrol. Perulangan digunakan apabila dalam program diperlukan untuk melakukan suatu hal yang sama beberapa kali, melakukan suatu hal secara bertahap, yang setiap tahap sebenarnya memiliki langkah yang sama dan mengakses sekumpulan data dalam suatu struktur data, misalnya: List, Tuple, Queue, Stack dan beberapa struktur data lainnya.

Pada Python, perulangan dapat dilakukan dengan menggunakan for, while maupun dengan cara rekursif. Untuk pertemuan ini, dibahas mengenai perulangan for dan while.

B. Bentuk Perulangan for

Salah satu perulangan pada Python adalah for dan while. Perulangan for biasanya digunakan dalam kondisi-kondisi sebagai berikut:

- Jumlah perulangan sudah diketahi sejak awal. Misalnya akan dilakukan pembacaan data dari 10 file teks. Walaupun setiap file teks memiliki isi yang berbeda, tetapi membaca file teks secara umum tetap sama. Pembacaan akan dilakukan dari file pertama, kedua, ketiga, dan seterusnya sampai file ke-sepuluh.
- Perulangan terjadi karena operasi yang sama pada suatu rentang data atau nilai. Misalnya dalam mencari jumlah dari 100 bilangan pertama, maka secara berturut-turut dilakukan penjumlahan 1 + 2 + 3 + ... + 100. Berarti dilakukan dalam rentang mulai dari 1 sampai 100.

Perulangan for pada rentang tertentu akan lebih mudah dilakukan jika menggunakan bantaun fungsi range(), silahkan liat contoh di bawah:

- range(stop). Digunakan agar menghasilkan rentang dari 0 sampai stop-1. Misalnya range(10), berarti menghasilkan rentang 0-5.
- range(start, stop, [step]). Digunakan agar menghasilkan rentang dari start-stop dengan peningkatan jumlah step.

Berikut ini adalah contoh program untuk menampilkan bilangan dari 1-100 dengan mengugunakan for dan range():

```
for i in range(1, 101):
    print(i)
```

Pada program tersebut, perulangannya dimulai dari 1 (start) sampai 101 (stop-1) dengan langkah 1 (default step adalah 1). Kemudian variabel `i` digunakan sebagai counter, di mana nilai `i` akan naik secara berurutan sesuai dengan nilai yang dihasilkan dari fungsi range() tersebut. Untuk kasus di mana tidak diperlukan counter, maka bentuk perulangan bisa seperti berikut ini:

```
for _ in range(1, 101):
    print('Hello World')
```

Program tersebut akan menampilkan tulisan Hello World sebanyak 100 kali, yang tidak membutuhkan nilai dari suatu counter.

Step negatif

Jika bisanya perulangan digunakan untuk menghitung dari nilai besar kecil, maka ada juga perulangan yang di gunakan untuk menghitung dari nilai ke kecil hingga nilai yang terbesar. Lihat lah contoh di bawah ini:

```
for i in range (100, 1, -2):
    print (i)
```

Program tersebut akan menampilkan bilangan genap mulai dari 100, 98, 96, 94, ..., sampai 2.

C. Bentuk Perulangan while

while biasanya digunakan pada kondisi di mana jumlah perulangan belum diketahui sebelumnya. Bentuk perulangan while secara umum adalah sebagai berikut:

```
<start>
while <stop>:
    operation
    operation
    <step (optional)>
```

Berikut ini adalah contoh perulangan dengan menggunakan while yang digunakan untuk memastikan input yang dimasukkan oleh pengguna adalah bilangan ganjil:

```
bilangan = 0
genap = False
while genap == False:
    bilangan = int(input('Masukkan bilangan ganjil: '))
    if bilangan % 2 == 1:
        genap = True
print(bilangan, 'yang anda masukkan adalah bilangan ganjil')
```

Output yang dihasilkan oleh program dapat dilihat pada Gambar 1.1. Awalnya memasukkan bilangan genap, tetapi program terus meminta pengguna memasukkan bilangan ganjil. Program berhenti setelah pengguna memasukkan 9, yang merupakan bilangan ganjil. Kasus ini sangat sesuai jika menggunakan perulangan while, karena tidak diketahui sampai berapa kali pengguna memasukkan bilangan ganjil yang tidak sesuai dengan permintaan.

```
Masukkan bilangan ganjil: 2
Masukkan bilangan ganjil: 4
Masukkan bilangan ganjil: 6
Masukkan bilangan ganjil: 8
Masukkan bilangan ganjil: 9
9 yang anda masukkan adalah bilangan ganjil
```

Gambar 1.1: Penggunaan while untuk mengambil input dari pengguna sampai sesuai dengan permintaan

D. Penggunaan Break dan Continue

Perulangan dapat dikontrol dengan menggunakan break dan continue. Secara umum break digunakan untuk menghentikan perulangan, sedangkan continue digunakan untuk melanjutkan perulangan ke iterasi berikutnya. Perhatikan program berikut ini yang akan menampilkan bilangan dari 1 sampai 10:

```
for i in range(1, 11):
    print(i)
print('Selesai')
```

Program tersebut akan menampilkan bilangan 1 sampai 10, kemudian pada baris terakhir muncul tulisan 'Selesai'. Jika diinginkan perulangan yang seharusnya sampai 10, dihentikan saat mencapai 5, maka diperlukan break dengan kondisi seperti program berikut ini:

```
for i in range(1, 11):
    if i == 5:
        break
    else:
        print(i)
print('Selesai')
```

Pada saat nilai i=5, maka boolean expression dari i==5 akan menghasilkan nilai True, program akan menjalankan instruksi break. Maka pada akhir program akan menampilan tulisan 'Selesai'. Continue digunakan pada saat diperlukan untuk melewati tahap perulangan sekarang, langsung dilanjutkan ke tahap/iterasi perulangan berikutnya. Sebagai contoh, berikut ini adalah program yang menampilkan angka dari 1 sampai 10, tetapi diperlukan untuk tidak menampilkan angka 6:

```
for i in range(1, 8):
    if i == 4:
        continue
    else:
        print(i)
```

```
print('Selesai')
```

Output yang dihasilkan dari contoh penggunaan continue dapat dilihat pada Gambar 1.2. Program menampilkan angka dari 1 sampai 7, tetapi melewatkan angka 4.

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\
1
2
3
5
6
7
Selesai
```

Gambar 1.2: Penggunaan continue untuk melewatkan iterasi saat counter(i) bernilai 6.

E. Konversi dari Bentuk for Menjadi Bentuk while

Bentuk perulangan for sebagian besar dapat dikonversi menjadi bentuk while. Beberapa hal yang ada di bentuk for dan while adalah sebagai berikut:

- Harus ada nilai awal, untuk memulai perulangan.
- Harus ada nilai akhir, untuk mengakhiri perulangan.
- Harus ada langkah, agar iterasi dari nilai awal bisa terus berjalan sampai mencapai nilai akhir.

Misalkan ada perulangan for seperti berikut:

```
for i in range(1, 11):
    print(i)
```

Maka dapat diidentifikasi bahwa perulangan tersebut dimulai dari 1, berakhir di 10, dengan step adalah 1. Konversi ke bentuk while dapat dilakukan dengan mudah, menghasilkan perulangan while berikut ini yang menghasilkan output yang sama:

```
i = 1                //awal
while i <= 10:       //kondisi akhir
    print(i)
    i = i + 1        //step
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

Tulis jawaban anda untuk soal nomor 1 di sini. Hapus paragraf ini.

```python
def perkalian(a, b):
    hasil = 0
    proses = ""

    for i in range(a):
        hasil += b
        proses += str(b)
        if i < a - 1:
            proses += " + "
    print(f"{a} x {b} = {proses} = {hasil}")

angka1 = int(input("Masukkan angka pertama: "))
angka2 = int(input("Masukkan angka kedua: "))
perkalian(angka1, angka2)
```

Penjelasan:

Program Python di atas dibuat untuk menghitung perkalian dua bilangan menggunakan penjumlahan berulang, bukan operator * biasa. Fungsi perkalian(a, b) menerima dua parameter, a dan b. Di dalam fungsi, terdapat variabel hasil yang awalnya diisi 0 untuk menampung hasil perkalian, dan variabel proses yang berupa string kosong untuk mencatat langkah-langkah penjumlahan. Program kemudian melakukan perulangan sebanyak a kali. Pada setiap iterasi, b ditambahkan ke hasil, dan nilai b juga dicatat ke dalam string proses. Agar proses penjumlahan terlihat rapi, program menambahkan tanda “+” di antara angka-angka, kecuali setelah angka terakhir.

Program meminta pengguna memasukkan dua angka melalui input() dan mengubahnya menjadi bilangan bulat dengan int(). Kemudian fungsi perkalian dipanggil dengan kedua angka itu sebagai parameter. Setelah perulangan selesai, fungsi mencetak hasil dalam format yang mudah dibaca, misalnya 6 x 5 = 5 + 5 + 5 + 5 + 5 + 5 = 12.

Output:

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrog
Masukkan angka pertama: 6
Masukkan angka kedua: 5
6 x 5 = 5 + 5 + 5 + 5 + 5 + 5 = 30
PS D:\Semester_2\Praktikum Algoritma dan Pemrog
Masukkan angka pertama: 7
Masukkan angka kedua: 10
7 x 10 = 10 + 10 + 10 + 10 + 10 + 10 + 10 = 70
```

SOAL 2

```python
def ganjil(bawah, atas):
    if bawah < atas:
        for i in range(bawah, atas + 1):
            if i % 2 != 0:
                print(i, end=", ")
    else:
        for i in range(bawah, atas - 1, -1):
            if i % 2 != 0:
                print(i, end=", ")

bawah = int(input("Masukkan batas bawah: "))
atas = int(input("Masukkan batas atas: "))
ganjil(bawah, atas)
```

Penjelasan:

Program Python ini dibuat untuk menampilkan deret bilangan ganjil di antara dua batas yang dimasukkan pengguna. Fungsi ganjil menerima dua parameter, yaitu bawah dan atas. Jika batas bawah lebih kecil dari batas atas, program akan menggunakan perulangan for dari bawah sampai atas dan memeriksa setiap angka dengan operator modulo % untuk menentukan apakah

angka tersebut ganjil (i % 2 != 0). Setiap bilangan ganjil yang ditemukan dicetak dengan print, dan end=", " digunakan agar semua angka dicetak dalam satu baris yang dipisahkan koma.

Sebaliknya, jika batas bawah lebih besar dari batas atas, program akan melakukan perulangan mundur dari bawah ke atas menggunakan range(bawah, atas - 1, -1). Prinsip pengecekan ganjil dan pencetakan sama seperti sebelumnya. Akhirnya, program meminta input dari pengguna untuk batas bawah dan atas, mengubahnya menjadi integer, lalu memanggil fungsi ganjil dengan nilai tersebut.

Output:

```
Bawah = 10
Atas = 30
11, 13, 15, 17, 19, 21, 23, 25, 27, 29
PS D:\Semester_2\Praktikum Algoritma da
Bawah = 97
Atas = 82
97, 95, 93, 91, 89, 87, 85, 83
```

SOAL 3

```python
def ips(x):
    qwe = 0
    for i in range(1,x+1):
        nilai = str(input(f"Nilai MK {i}: "))
        if nilai == "A":
            qwe += 4
        elif nilai == "B":
            qwe += 3
        elif nilai == "C":
            qwe += 2
        elif nilai == "D":
            qwe += 1
        else:
            qwe += 0
    qwe = qwe / x
    return f"Nilai IPS anda semester ini {qwe:.2f}"

print("Program penghitung IPS Mahasiswa")
x = int(input("Berapa jumlah mata kuliah? "))
print(ips(x))
```

Penjelasan:

Program Python ini dibuat untuk menghitung Indeks Prestasi Semester (IPS) seorang mahasiswa berdasarkan jumlah mata kuliah dan nilai yang diperoleh. Fungsi ips(x) menerima jumlah mata kuliah x sebagai parameter. Di dalam fungsi, variabel qwe digunakan untuk menampung total bobot nilai. Program melakukan perulangan dari 1 sampai x, meminta pengguna memasukkan nilai setiap mata kuliah (A, B, C, atau D). Nilai tersebut kemudian diubah menjadi bobot angka: A = 4, B = 3, C = 2, D = 1, sedangkan nilai selain itu dianggap 0. Semua bobot dijumlahkan ke variabel qwe.

Setelah semua nilai dimasukkan, total bobot dibagi jumlah mata kuliah x untuk menghitung IPS. Fungsi kemudian mengembalikan hasil IPS dengan format dua desimal agar lebih rapi. Program utama menampilkan judul, meminta pengguna memasukkan jumlah mata kuliah, lalu memanggil fungsi ips(x) dan menampilkan hasilnya.

Output:

```
Program penghitung IPS Mahasiswa
Berapa jumlah mata kuliah? 6
Nilai MK 1: A
Nilai MK 2: B
Nilai MK 3: C
Nilai MK 4: A
Nilai MK 5: D
Nilai MK 6: C
Nilai IPS anda semester ini 2.67
```

```
Program penghitung IPS Mahasiswa
Berapa jumlah mata kuliah? 6
Nilai MK 1: A
Nilai MK 2: A
Nilai MK 3: A
Nilai MK 4: A
Nilai MK 5: A
Nilai MK 6: A
Nilai IPS anda semester ini 4.00
```